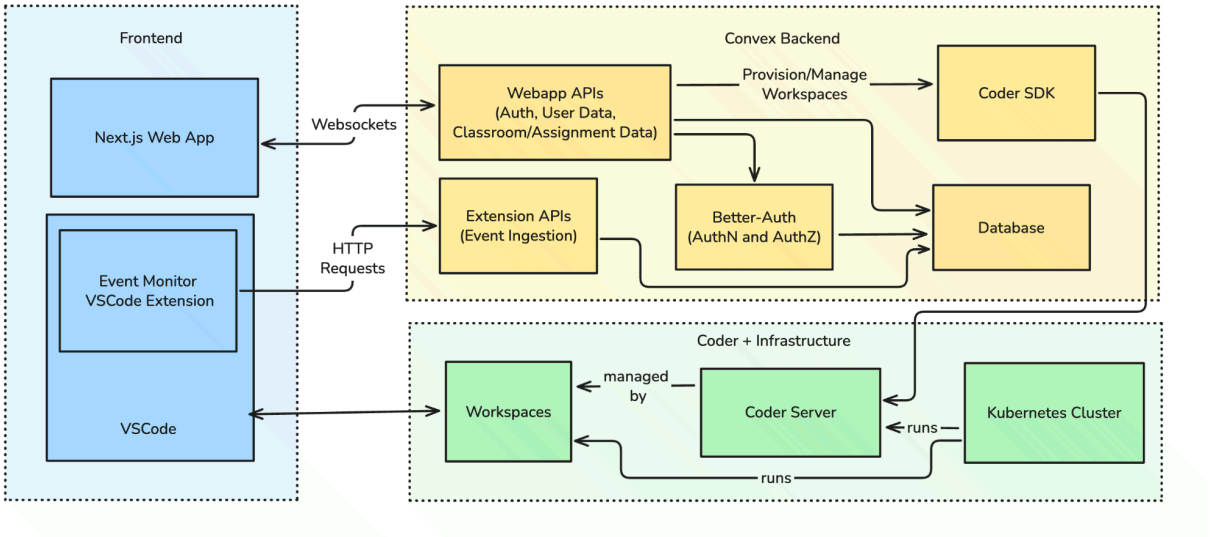
Leopard - Design, Validation and Verification Results

Group 18 - Krish Krish, Rachelle De Man
Felix Fong, Arian Ahmadinejad, Timothy Leung
Version 1
COMPSCI 4ZP6
Dr. Mehdi Moradi
Jan 8, 2026

Purpose Statement

Leopard is a web based IDE platform that allows students to complete assessments and instructors to manage assignments. It provides standard code environments for students to code with, a granular replay viewer of the students actions, and a UI to manage all their assignments and classes. From the instructors perspective, Leopard provides a way to assign coursework while finding key areas students struggle with. With Leopard, we enable live session replay where instructors can view each student's work. The purpose of this document is to outline the technical design for Leopard, along with key decisions, tradeoffs, and a high level testing and validation plan for each component.

Component Diagram



Component <> Requirements Relationship

Component	SRS Requirements Met
Backend API & Database (Convex, Better-Auth)	**Functional:** **- P0:** Authentication & Authorization, with roles and resource access checks **- P0:** Versioned record of documents **- P0:** Instructors can view code submissions **- P1:** Classroom/Assignment grouping **- P1:** Replay support **- P2:** Student submissions can be viewed in (near) real-time **Non-Functional:** **- Performance:** sub 1s update when viewing live edits, supported by Convex websocket connections **- Security:** All data is accessed through SSL encrypted HTTPS or Websocket requests

Component	SRS Requirements Met
Coder Server + Coder VSCode Workspaces	**Functional:** **- P0:** Web-based IDE for students **- P1:** Students can execute their code **Non-Functional:** **- Editing Performance:** Editor latency is similar to local editor given ideal network conditions **- Code Execution Time:** Since editor is running in a full Linux environment, we achieve sub 60s execution times **- Look and Feel:** The IDE is a standard and modern development tool (VSCode) **- Isolation:** Each student gets a private container to work, containing damage from malicious code execution
Event Monitor VS Code Extension	**Functional:** **- P0:** User authorization; the extension only logs data for the logged in user by sending appropriate session tokens **- P0:** Versioned record of every document **- P1:** Replay support
Next.js Web Application	**Functional:** **- P0:** Students edit, submit, and manage assignments **- P0:** Instructors review submissions **- P0:** User authentication and authorization (supports auth flows and checks) **- P1:** Session replay viewing **- P1:** Classroom and assignment grouping/management **- P2:** Instructor can view a student session live in real time **Non-Functional:** **- Look & Feel:** Application will be simple, sleek, and lean onto existing understandings of other systems. It is tailored for educators and students and as such will have a feel similar to that of education platforms and popular IDEs (VSCode). **- Responsiveness:** Loading time between actions will be non-interruptive and clear feedback will be given upon taking an action. **- Consistency:** The web application uses consistent components fit within a design system.
Infrastructure (Kubernetes)	**Functional:** **- P1:** Students can execute their code (provides isolated compute environments) **Non-Functional:** **- Scalability:** Dynamic resource provisioning allows workspaces to spin up and down based on demand, reducing costs during idle periods **- Isolation:** Each student workspace runs in its own Kubernetes pod with dedicated resources, preventing interference between students

Component	SRS Requirements Met
	- Availability: Persistent volumes ensure student work survives pod restarts and infrastructure updates

Project Components

Backend / Database

We have designed our application around various key entities that represent the workflow of our application. A list of the key entities is described below. For a full database schema, please refer to the Appendix (Database Schema).

- Users
- Classrooms
- Assignments
- Workspaces
- Submissions
- Events
- Flags

Most of the backend API routes/operations can be categories into generic Create/Read/Update/Delete (CRUD) operation w.r.t. each of our key data objects that define an entity in the context of our application. Those are as follows:

Component	Operations
User/Profiles	- Get current user (auth.getCurrentUser) - Get profile (userProfile.getProfile) - Create profile with Role (userProfile.createProfile) - Update user Role (userProfile.updateRole)
Classrooms	- Get my classrooms - teacher (teacher.getMyClassrooms) - Get classroom details (teacher.getClassroomDetails) - Create classroom (teacher.createClassroom) - Update classroom (teacher.updateClassroom) - Delete classroom (teacher.deleteClassroom) - Get enrolled classrooms - student (classroom.getEnrolled) - Get available to enroll (classroom.getAvailableToEnroll) - Enroll in classroom (classroom.enroll) - Get pending enrollments (teacher.getPendingEnrollments) - Get enrolled students (teacher.getEnrolledStudents) - Update enrollment status (teacher.updateEnrollmentStatus) - Remove student (teacher.removeStudent)
Assignments	- Get assignments by classroom (teacherAssignments.getAssignmentsByClassroom) - Get assignment by ID (assignment.getById) - Get assignments by IDs (assignment.getByIds)

	- Create assignment (teacherAssignments.createAssignment) - Update assignment (teacherAssignments.updateAssignment) - Delete assignment (teacherAssignments.deleteAssignment)
Submission	- Get my submissions (submissions.getMySubmissions) - Get my submission for assignment (submissions.getMySubmissionForAssignment) - Get or create submission (submissions.getOrCreateSubmission) - Submit assignment (submissions.submitAssignment) - Unsubmit assignment (submissions.unsubmitAssignment) - Get submissions by assignment - teacher (teacherAssignments.getSubmissionsByAssignment) - Grade submission (teacherAssignments.gradeSubmission) - Create flag on submission (teacherAssignments.createFlag) - Get flags by submission (teacherAssignments.getFlagsBySubmission) - Delete flag (teacherAssignments.deleteFlag) - Mark submission (teacherAssignments.MarkSubmission) - Get Grade (submissions.getGrade)
Workspaces	- Launch workspace (assignment.launchWorkspace) - Creates/resumes Coder workspace - Ingest events from VS Code (extension.addBatchedChangesMutation) - Get replay events (replay.getReplay) - Granular changelog for a particular workspace

Beyond basic CRUD endpoints, we have a few key operations that work across various layers of our application. Those are as follows:

Launch Workspace for Coding

Students need to be able to launch their own personal VSCode instance to work on their assignments. This is done through the `assignment.launchWorkspace` API route.

Route	`assignment.launchWorkspace`
Implementation	This endpoint orchestrates and syncs our backend state with Coder using their SDK. It creates/fetches Coder user, gets/creates workspace, stops other workspaces, generates session key, stores active workspace in the database, and returns the workspace URL and session details so the frontend can redirect the user to their
API Input	`{ assignmentId: Id<"assignments"> }`
API Output	`{ workspaceUrl: string, coderUserSessionKey: string }`
Normal Behaviour	Launches or resumes a Coder workspace for the assignment, creating a Coder user if needed.

Undesired Behaviour	Undocumented or unexpected errors from Coder SDK, which can cascade to errors in our application.

Event Ingestion

Route	`extension.addBatchedChangesMutation`
Implementation	Uses better auth to authenticate the user workspace session. From the session, it derives the current active `workspaceId` and adds the events linked to this workspace in the database.
API Input	`{ changes: Array<{ eventType: string, timestamp: number, metadata: Record<string, any> }> }`
API Output	`void`
Normal Behaviour	Intended for use by the Event Monitor VSCode Extension. It batch inserts events for workspace activity tracking. The workspace is automatically inferred from the user authentication session.
Undesired Behaviour	- Fails to infer workspace - User events not related to any workspace - EventType is unsupported - Metadata does not match the eventType - Missing events from a workspace session due to network connection failure, causing the history to be out of sync

Replay Viewing

Route	`replay.getReplay`
Implementation	Uses the `workspaceId` to fetch the events associated with the workspace. These events are sorted in ascending order in terms of their timestamp field and returned back.
API Input	`{ workspaceId: v.id("workspaces") }`
API Output	`Array<{ eventType: string, timestamp: number, metadata: Record<string, any> }>`
Normal Behaviour	Fetches and aggregates the granular event history from the extension into one cohesive set of logs to be replayed.
Undesired Behaviour	Unmatched workspaceId returns empty event history. Network failure during ingestion can also create gaps between the replay.

Frontend

Normal Behaviour	The frontend of the application will be reliable, sleek, and follow the designated theming (more information in UI Design). Consistency will be kept through all pages through component reuse, and reliability will be maintained.
Implementation	React, TypeScript, and Tailwind CSS are used to create the frontend pages.
Potential Undesired Behaviour	Window resizing resulting in undesired layouts. The state of the frontend not reflecting current updates and showing stale data.

Event Monitor VSCode Extension

`src/apps/vscode-extension`		
Normal Behaviour		This extension will be installed into the editor for every user. It listens to every user's events and sends them to the backend in real time.
API contract	**Input**	`{ context: vscode.ExtensionContext }` The `context` is a collection of code editor utilities that are private to an extension.
	Output	The extension will mutate the `context` parameter to extend the behaviour of the code editor. It will also create side effects such as uploading real time user event data to the backend.
Implementation		Interfaces with vscode API to access user input. It uses the following APIs from vscode to extract user event data: `onDidChangeTextDocument` - granular edits to a text document `onDidAcceptCopy` - content that the user copied and where `onDidAcceptPaste` - content that the user pasted and where `onDidChangeTerminalState` - terminal commands that were executed All event data is sent to the backend via HTTP using the `extension.addBatchedChangesMutation` endpoint defined above.
Potential Undesired		- Failed to listen to an event from VSCode - Could not connect to the backend client (bad connection)

Behaviour	- Uploading to the backend fails - VSCode changes implementation details of their event listeners

Infrastructure

We use Kubernetes to manage our infrastructure. Most VS Code workspaces sit idle while students aren't working on their homework but with Kubernetes, we can spin resources up and down as needed, reducing costs. Coder.com is an open source tool that handles provisioning VSCode through an admin dashboard and API, which is essential for integration with the rest of our app. We also self-hosted the application for Leopard. Our architecture is as follows:

We use several Kubernetes namespaces to establish logical boundaries between different parts of the infrastructure, giving us separation of concerns:

Coder Namespace hosts the image provided by Coder.com alongside a PostgreSQL database for data persistence. Since each VSCode workspace needs to run on its own isolated compute environment, Coder provisions a Kubernetes pod for each workspace.

Workspaces Namespace is where these workspace pods live, isolated from the core Coder infrastructure. Each workspace pod has its own persistent volume (PVC) for storing student work, ensuring that files persist even when pods are scaled down to save resources. For communication across namespaces, we configure strict RBAC permissions (similar to folder and file permissions on Linux).

CertManager Namespace handles the last piece of infrastructure: secure external access. We created a Certificate Issuer pod that connects to the Let's Encrypt tool to generate SSL certificates for secure web connections. These certificates are stored in Kubernetes' cert-manager. The networking for our cluster is handled by Traefik Ingress, which reads certificates from cert-manager and provides secure access to resources. We have to use SSL since VS Code won't allow WebSocket connections over an unsecured connection.

An architecture diagram for our infrastructure is included in the appendix.

Design System UI

Following our name of Leopard, our main theming and colouring is based on snow leopards, using black, white, and light blues. Our priorities were simplicity, sleekness, and mimicking standardized UI. We want to leverage our users' understanding of different education systems to minimize the initial learning curve for use. We also do this within our coding environment by having the design be exactly like VS Code, a very popular IDE.

We have two main, role-based views. One is for students which allows them to view their classes, their assignments, and launch coding workspaces. Students can view assignment details as well as submit their work here. The other is for instructors to manage assignments, classrooms, etc.

Student View

From the student homepage students can view their assignments organized by course, as well as view available classrooms to join. There is a shortcut button to instantly launch the workspace, or the student can first view the assignment details. Students will have access in the header bar to go to their dashboard, their profile, or to sign out. (Student Homepage Image)

Within the assignment details, students will be able to see the assignment title, the due date, and the description. A launch workspace as well as submit button are here as well to allow the student to do their work. The student can exit this page by either clicking on the back to classroom button, or by using the top header navbar. (Student Assignment View Image)

Instructor View

Instructors will have access to a dashboard where they can view and manage their classrooms. Similarly to students they will have access in the header bar to go to their dashboard, their profile, and to sign out, but with the addition of the ability to view their classes.

Upon selecting a class the instructor will be able to view the enrolled students, assignments, pending join and pending join requests. Below this shows each of these sections in detail. There are also buttons to allow the instructor to add new assignments to their course. Depending on the role of the instructor (head instructor, teaching assistant) different functionalities are available. For instance, a teaching assistant cannot delete a course. (Instructor Course View Image)

Upon viewing an assignment in detail the instructor can see their description, due date and time, number of student submissions, and number of graded submissions. You can also edit anything within the assignment from this page. Furthermore, student submissions can be accessed from here as well. This is done on purpose to keep the assignments themselves as the center. (Instructor Assignment View Image)

We wanted to leverage our users' understanding of video players for the session replay. To do so we mimic video playback UI by using a scrubbable bar. The position of the selector within the bar coincides with how far along in the replay is being shown. This was done to ensure a clear mapping between the scrubber position and the replay. The user can interact with the replay by using the blue scrubber (when it is hovered the end position circle expands slightly), or by clicking the "play" button below the scrubber to have the replay shown automatically. Yellow was used for potential flagged areas due to the contrast it has with our main colour scheme, signalling it as an important point. The button to return to the current assignments replays is at the tip above the viewer to provide an easy way to switch between replays of an assignment. (Replay Viewer Image)

The entire design process and prototypes on Figma can be found under Figma Prototypes in the Appendix.

Extended Validation and Verification Plan

The test runner results can be viewed at:
https://github.com/Beanie-Brick-Band/leopard/actions/runs/23958456466/job/69882401180

Backend/Database

User and Profile Management tests are as follows:

- `getCurrentUser()` function unit tests will ensure a valid session returns a user object, an invalid session returns null / throws AuthenticationError, and an expired token returns null / throws SessionExpiredError.

- The `getProfile(userId)` function will have a test that verifies that the userId returns a profile object (no sensitive fields), and that a non-existent userId returns null / throws NotFoundError.

- The `createProfile(data)` function will have a test ensuring valid data creates a profile and returns profileId. There is also a unit test that checks if a duplicate email is provided, ensuring it will throw a ConflictError in that case.

- The `updateRole(userId, role)` function will have tests checking that when it is called by an admin it updates the role, and when called by a non-admin it throws a PermissionError.

Classroom and Assignment Management tests are as follows:

- The `createClassroom()` function will ensure only users with the instructor role can create a classroom, otherwise a `PermissionError` is thrown. The `getMyClassrooms()` function is similar but also checks only the instructors' own classrooms are returned. The `deleteClassroom(classroomId)` function will have tests ensuring if there are active students it is only soft deleted.

- The `enroll(classroomId)` function will be tested so that if the classroom is available it creates a pending enrollment. The `updateEnrollmentStatus(enrollmentId)` function will have tests ensuring it only works for an owner instructor role, and has approval and denial capabilities.The `removeStudent(studentId)` similarly tests that it only works for owner instructor roles.

- For Assignment Management each function (`createAssignment()`, `updateAssignment()`, `deleteAssignment()`, `getAssignmentsByClassroom(classroomId)`, and `getAssignmentById(assignmentId)`) will do checks ensuring access is blocked if the user does not have an approved role for that action. Tests will also be done to ensure the functions work as expected. Submission Management is very similar, with those functions (`getOrCreateSubmission(assignmentId)`, `submitAssignment(assignmentId)`, `unsubmitAssignment(submissionId)`, `getMySubmissions()`, and `gradeSubmission(submissionId)`) having proper role unit tests and behaviour tests.

Workspace Management will have:

- Unit tests for `launchWorkspace(assignmentId)` to create a workspace and return workspace URL upon first call, resume the workspace if it already exists, and stop previous active workspace if one exists.

- There will also be unit tests for when the Coder API fails, or an unassigned assignment attempts to have a workspace launched. The `setUserActiveWorkspace(workspaceId)` and `getUserActiveWorkspace()` will also have unit tests for expected behaviour.
- Unit tests will also be created for flag management. It will have tests for cases such as missing authentication and permission errors.

Backend/Database Test Results

We implement this test suite across our backend files. These tests involve checking the correct database tables are updated on each event, and ensure only users with the correct permissions can access the different endpoints. The results are as follows:

Test Suite	**Tests**	**Status**
web.user.test.ts	5	✅ PASS
web.classroom.test.ts	13	✅ PASS
web.teacher.test.ts	13	✅ PASS
web.submission.test.ts	16	✅ PASS
web.teacherAssignments.test.ts	19	✅ PASS
storageKeys.test.ts	4	✅ PASS
minio.test.ts	9	✅ PASS
starter-code.integration.test.ts	4	✅ PASS
TOTALS	**83**	✅ **83/83 PASS**

Student Classroom View

We will create integration tests to ensure students can open their classes, open their assignments, and are able to view all related information. Accurate due dates and submissions status will also be tested.

We will also validate by manual testing to ensure their VSCode instance is opened in sub 60 seconds, and that the student has access to only what they are permitted.

Unit tests will be made checking the submission flow, ensuring the latest changes are submitted, resubmissions function properly, and submitted status is accurate and updated.

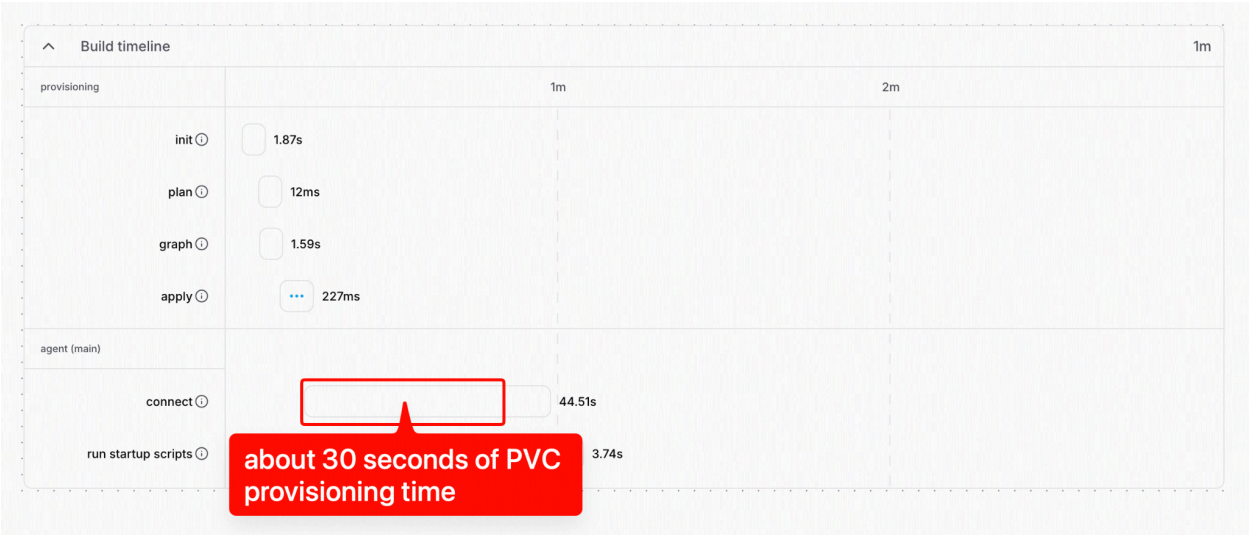
Student Classroom View Test Results

Our unit tests verify the functionally mentioned above.

Test Suite	Tests	Status
web.classroom.test.ts	13	✅ PASS
web.submission.test.ts	16	✅ PASS
storageKeys.test.ts	4	✅ PASS
TOTALS	**33**	✅ **33/33 PASS**

For workspace launch time, Coder actually tracks the time interval taken by each section of the deployment process. If using ephemeral workspaces where a PVC storage doesn't need to be mounted by a kubernetes provider, we get deployments in the 20-30 seconds range consistently. Breakdown of time spent on each component can be viewed in the screenshot below.

However, for persistent storage we rely on our Kubernetes provider Civo to provision a PVC volume. This is a crucial bottleneck in our system that prevents us from achieving sub 60 second workspace creation. We tested this on a different kubernetes deployment on Oracle Cloud and the PVC creation was much faster. Due to limitations of cost, we are opting to forgo this requirement and accept the cost-speed tradeoff for our project. As we can see in the screenshot below there's an added 30 second to agent startup time. Coder doesn't have context on why this is happening but taking the delta of when this process starts and kubernetes instance creation we can infer time taken by PVC provisioning.



Instructor Classroom View

Integration tests will be created to ensure instructors are able to view all details about their classes including students, assignments, and enrollment requests.

Within the assignment page, unit tests will test for proper assignment submission statistics, editing functionality, and grading functionality.

We will also create tests that check if the instructor has proper access to features depending on their role. One such example is ensuring only the main instruction can delete the course or assignments, and Teaching Assistant roles cannot take that action.

Instructor Classroom View Test Results

As visible below, we created unit tests to ensure our requirements such as assignment, enrollment, and student management. We ensured submission statistics, editing and grading functionality were complete.

Test Suite	Tests	Status
web.teacher.test.ts	13	✅ PASS
web.teacherAssignments.test.ts	19	✅ PASS
TOTALS	**32**	✅ **32/32 PASS**

Replay Viewer

Unit tests verify base functionality is reliable regarding database connection:

- Verify proper replay is fetched when requested
- Verify replay contains every granular change
- Verify strict schema support on component input so we can render the replay

Unit tests verify base functionality regarding UI:

- Verify all received change states can be accessed
- Verify the correct state of replay is always maintained as the scrubber is moved around

Unit tests will also be done to check niche behaviour results in the proper replay display. Some examples of the behaviour to be tested are multiline editing, shortcut word swap, editing all occurrences of a word, line swapping, inserting above the cursor, and line duplication. These are some advanced cases of editing that the replay must support.

Manual performance validation will be conducted to ensure that fetching the data to replay happens quickly, and moving between replay positions has minimal visual lag and has a high FPS on modern computer systems.

Replay Viewer Test Results

Manual performance testing was completed to ensure a positive user experience as well as no visual bugs.

We created `scrubber.test.tsx` with extensive unit tests including our defined multiline editing, shortcut word swap, editing all occurrences of a word, line swapping, inserting above the cursor, and line duplication. These tests all pass. Here is a summary:

Test Case	**Line**	**Status**
Multiline editing	112-123	✅ PASS
Shortcut word swap	149-163	✅ PASS
Editing all occurrences of a word	182-191	✅ PASS
Line swapping	230-244	✅ PASS
Inserting above the cursor	317-337	✅ PASS
Line duplication	645-696	✅ PASS

Event Ingestion

Event ingestion will be validated with unit tests backed by a mock database.

Tests will first verify that the workspace is correctly inferred from the authenticated session. To verify this, we can create a mock session that includes a valid workspace. We can then provide the API a well formed event payload, resulting in the ingestion logic writing a record containing the expected `workspaceId`. A complementary test will use a mock session with no active

workspace and assert that ingestion returns a clear “cannot infer workspace” error and does not write any events.

We will then validate event type handling. For unsupported `eventType` values, ingestion should reject the payload with an appropriate error and avoid persisting it. For all required `eventType`s, tests will confirm that they are accepted and correctly stored in the mock database.

Finally, we will enforce the contract between `eventType` and `metadata`. For each supported event type, tests will pass matching metadata and assert that ingestion succeeds, and then pass mismatched metadata and assert that ingestion returns a clear validation error. This ensures that only structurally valid, replayable events are persisted.

Event Ingestion Test Results

These test results are the same as our backend tests mentioned above. All tests pass from the backend test suite.

Event Monitor VS Code Extension

The VSCode extension has 3 undesired behaviours:

- Failure to listen to an event from vscode
- Uploading to the backend fails
- Could not connect to the backend client (bad connection)

All these pain points occur during an interaction on the boundary between components. As such, the test methodology used here will be integration tests.

The integration test will load the extension into an instance of VSCode. The extension will in turn load a Convex Client connected to a mocked database. The test will be as follows:

We will simulate each of the events that the extension is expected to handle (`onDidChangeTextDocument`) in our VSCode instance. The events `onDidAcceptCopy`, `onDidAcceptPaste`, `onDidChangeTerminalState` did not need to be implemented:

- `onDidAcceptCopy`, `onDidAcceptPaste` were deprioritized as flagging user events was decided to be a deprioritized feature
- `onDidChangeTerminalState` was already implemented as part of `onDidChangeTextDocument`

When an event is handled, the extension will upload it to the mocked database. We can then verify if the extension could listen to vscode by checking that it was uploaded to the database. This method also verifies the success of uploading to the backend.

In case of a bad connection, the extension will be tested to ensure it has a proper error message. To do this, mock the Convex Client again to simulate transient network failures. One of the relevant events will then be simulated, and the error from the extension can then be captured and verified.

Event Monitor VS Code Extension Test Results

There are 2 unit tests, one to test the `onDidChangeTextDocument` behaviour. This is an overarching unit test that tests the success of submitting the event as well as batch submissions when several events have to be submitted at once.

The other unit test is to test failed database submissions in case of a bad connection as described above.

Test Suite	Tests	Status
extension.test.ts	2	✅ PASS
TOTALS	**2**	✅ **2/2 PASS**

Full System Validation

Due to the inherently complex nature of all the components and how they inter-connect, we will have to perform manual human testing of the user journey for different roles (instructor, student) to verify that all actions act as expected with the full system assembled.

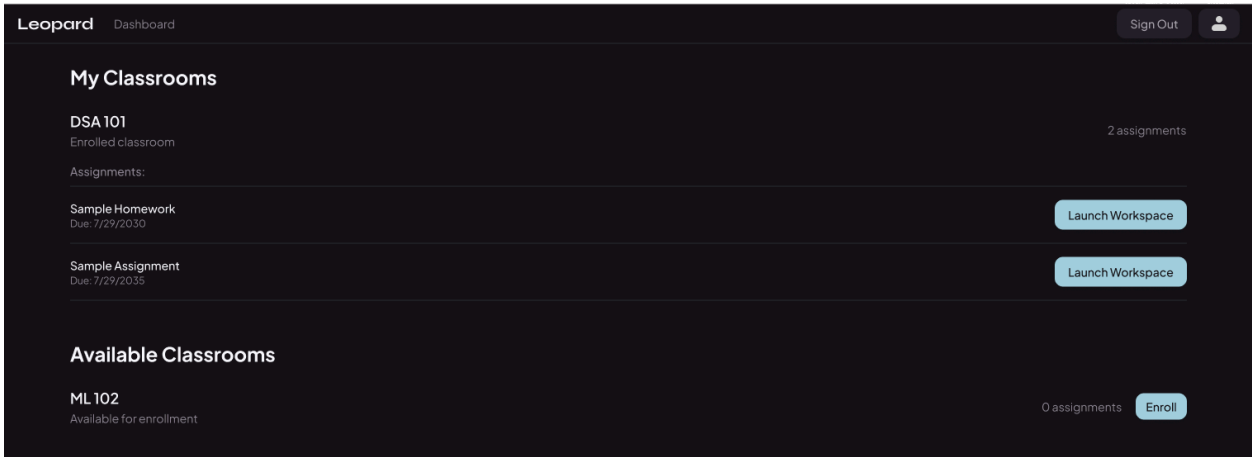
Full System Validation Test Results

We conduct this mostly as a manual inspection, as well as some UI component tests.

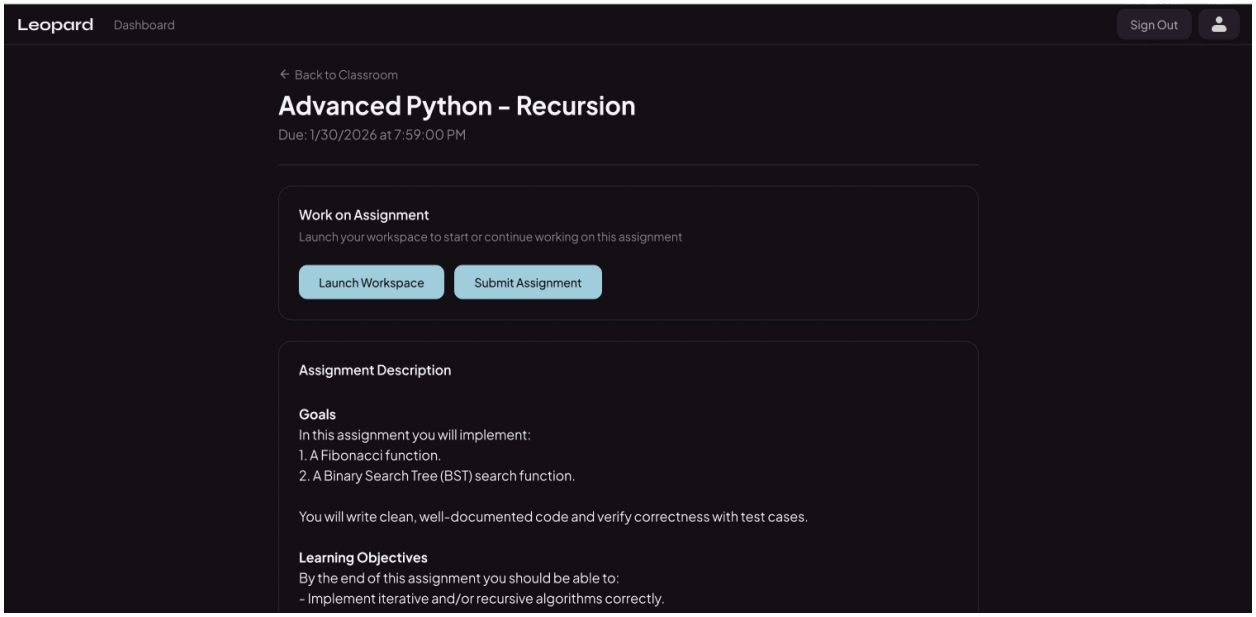
Test	Methodology	Status
UI Component Testing	Unit tests for UI components, run in CI.	✅13/13 Tests Pass
Student View Access	Student actions are visible and performable.	✅Confirmed by extensive manual inspection
Student Coder Access	Students can write and submit code from Coder	✅Confirmed by manual inspection
Teacher View Access	Teachers' actions are visible and performable.	✅Confirmed by extensive manual inspection
Teacher Replay Access	Teachers are able to see everything students did in their submission	✅Confirmed by manual inspection

Appendix

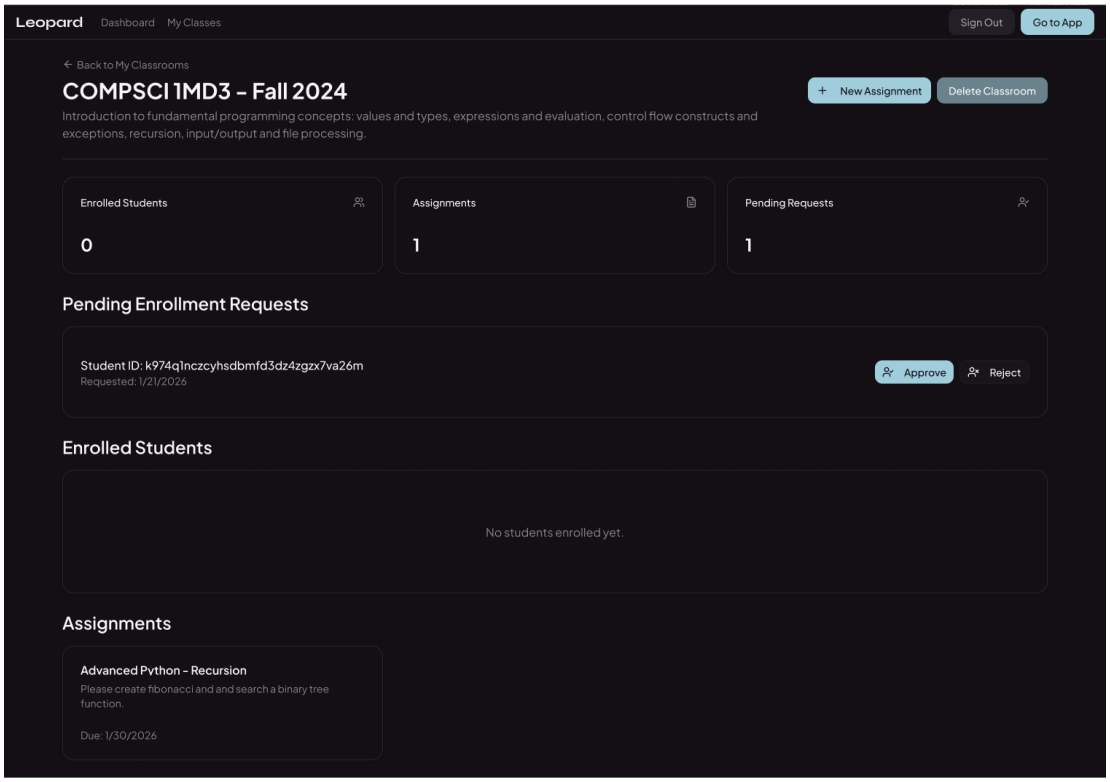
Student Homepage



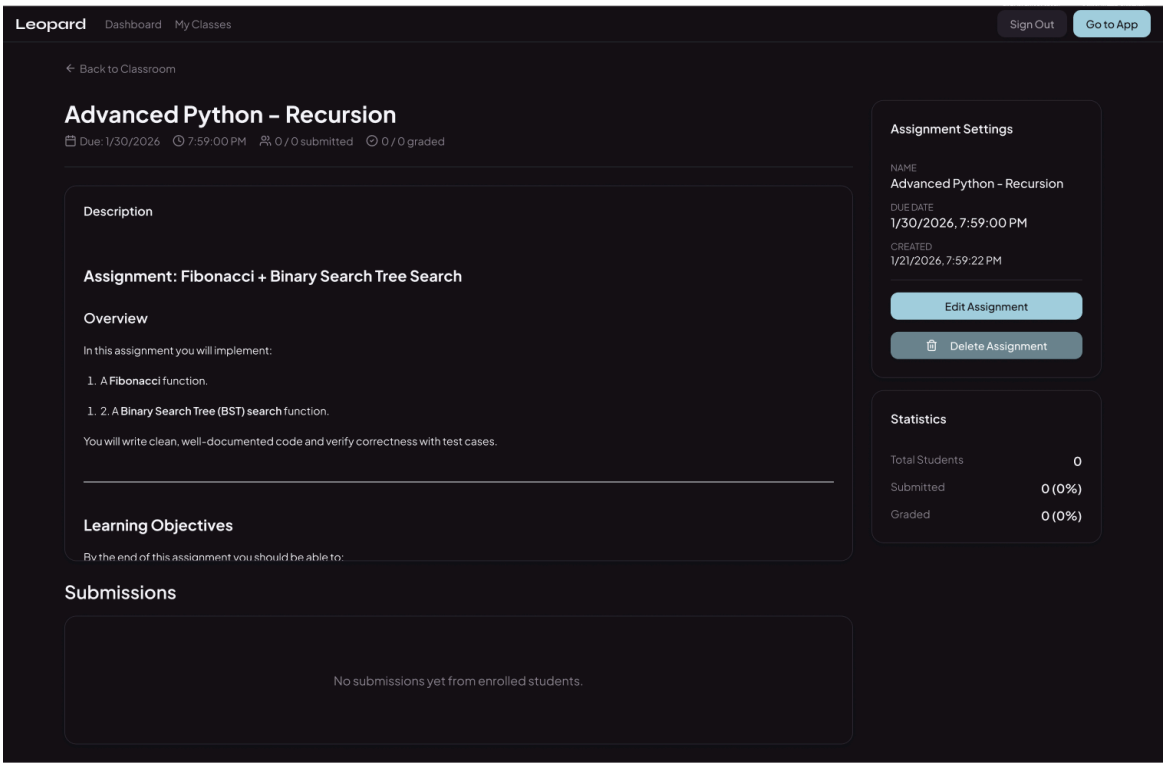
Student Assignment View



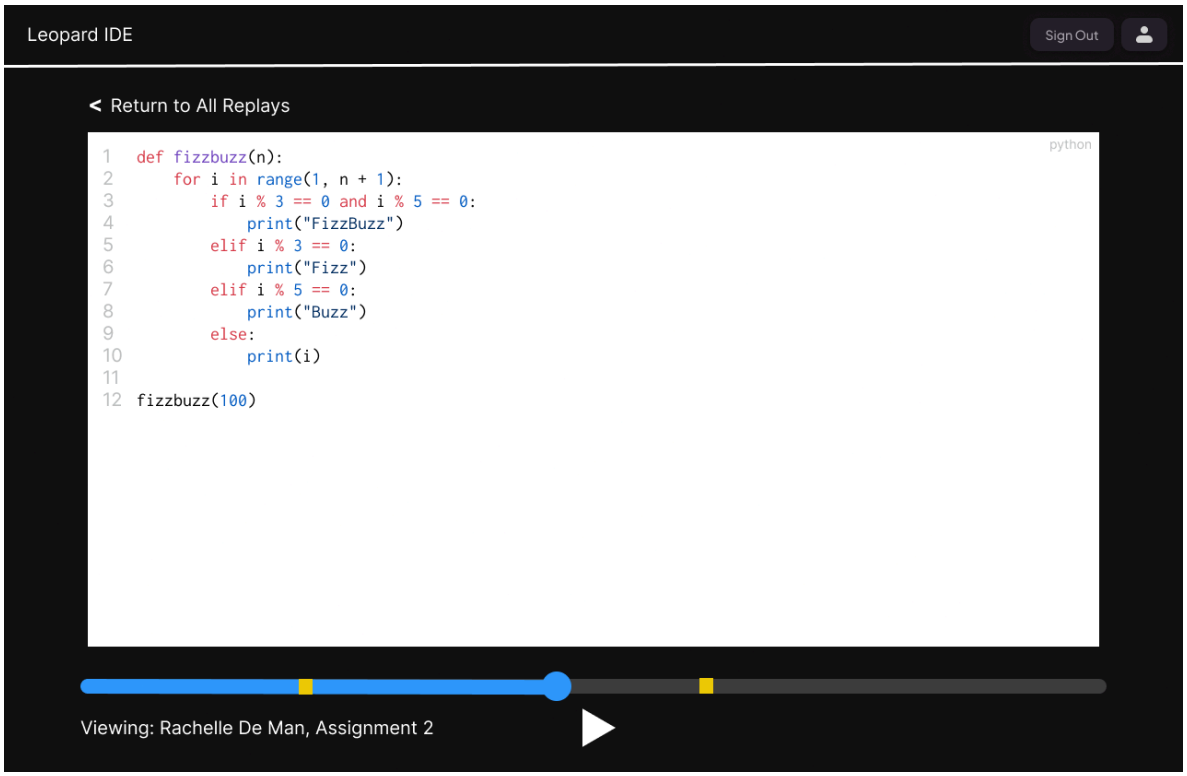
Instructor Course View



Instructor Assignment View



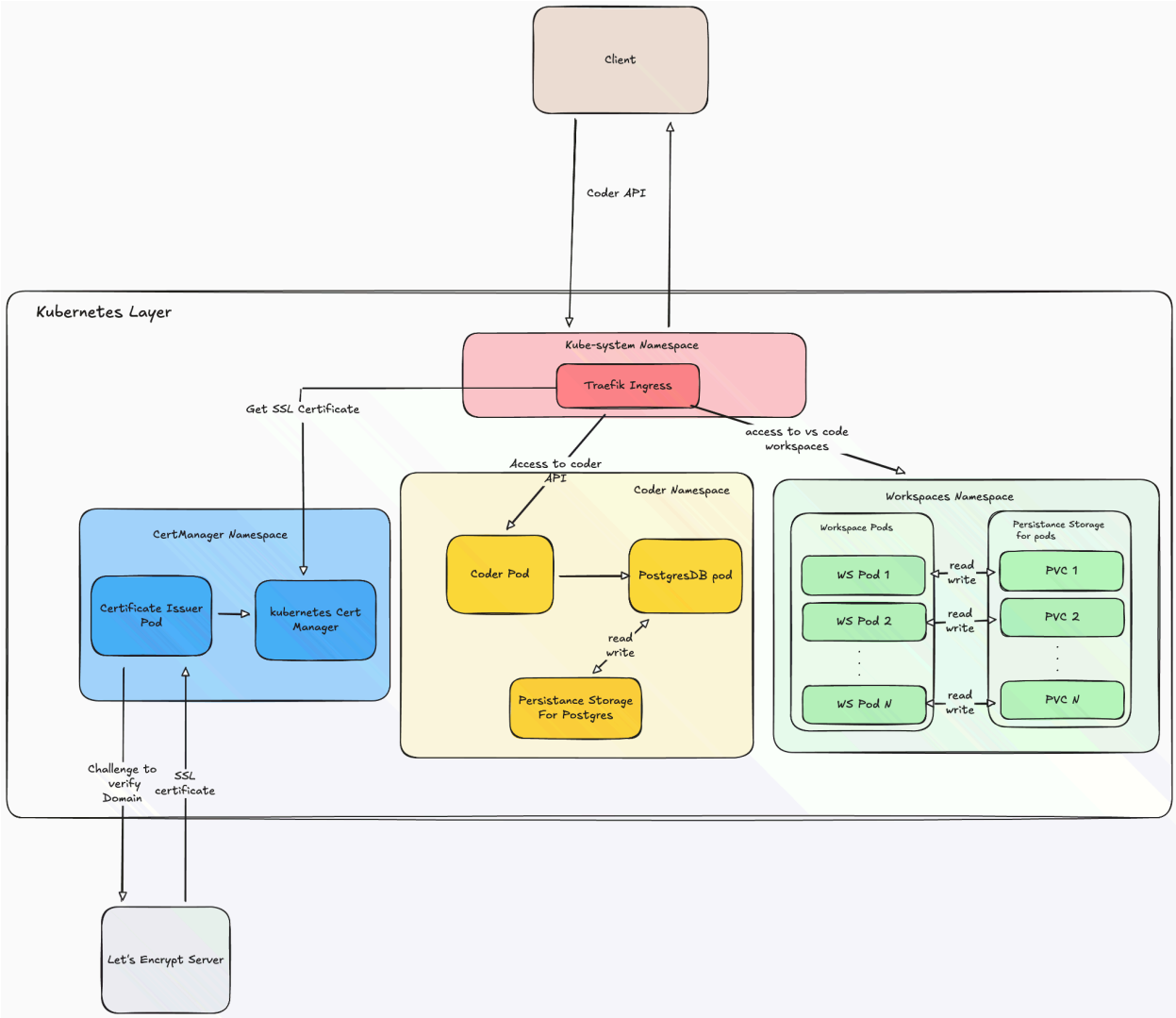
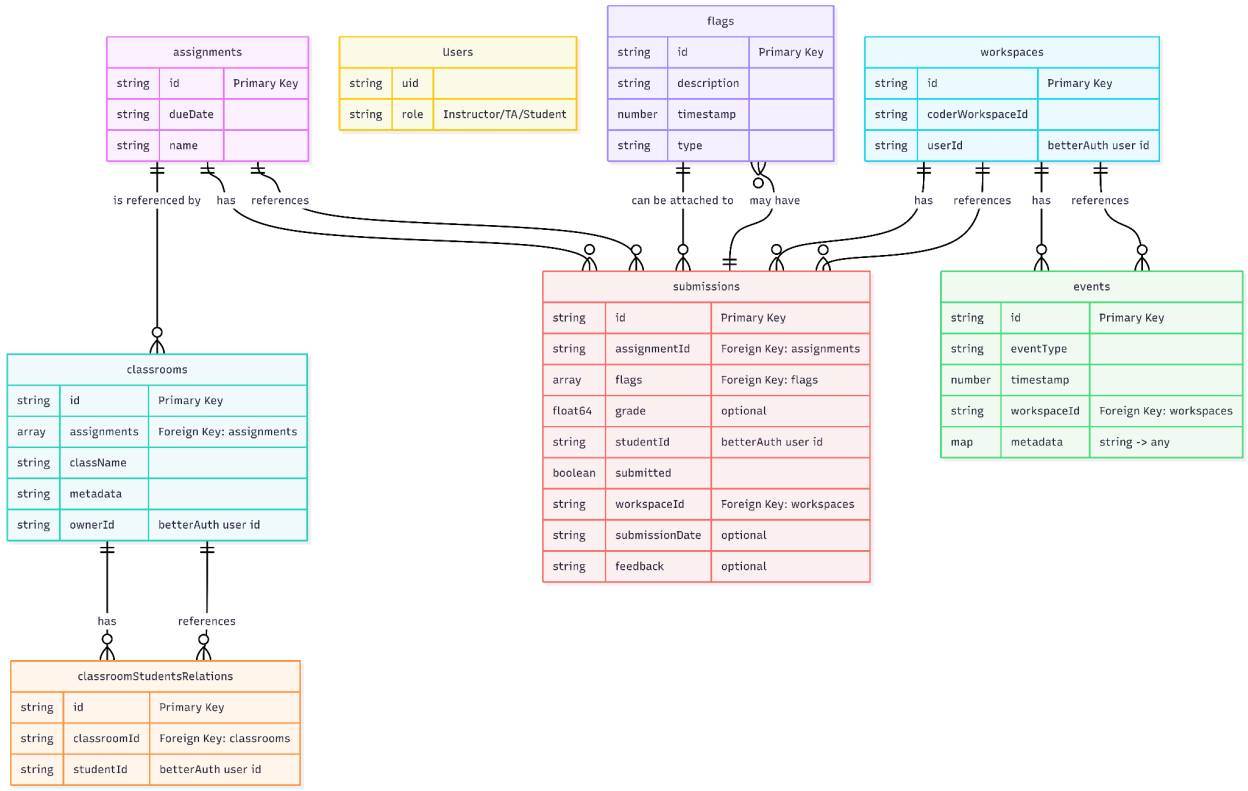
Replay Viewer View



Figma Prototypes

https://www.figma.com/design/06Nmmjsdmpx3FvgfaQTzrF/beanie-brick-band?node-id=0-1&t=Gu34Jzt9K9Phc3gy-1

Infrastructure Architecture Diagram



Database Schema